SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: MLA0202 COURSE NAME: Fundamentals of Machine Learning

---

COURSE OUTCOME COVERED

CO2: Apply supervised learning algorithms for regression, classification, and predictive modeling tasks to solve structured data problems in practical applications. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Scenario-Based

Code of Conduct:

I ***Kruthika Sre S / 192525360*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kruthika Sre

Annexure A

Scenario-Based

Q1	Apply a linear regression model to predict apartment rent based on floor area, number of rooms, and location in a real-estate scenario.	5 Marks	6 min
Q2	Use a linear classification model to categorize emails as spam or non-spam using word frequency features in a mail service system.	5 Marks	6 min
Q3	Apply the Naïve Bayes algorithm to predict whether a patient has flu based on symptoms such as fever, cough, and headache.	5 Marks	6 min
Q4	Use discriminant functions to classify loan applicants into eligible and non-eligible groups based on income and credit score.	5 Marks	6 min
Q5	Apply a probabilistic generative model to classify handwritten digits in an optical character recognition system.	5 Marks	6 min
Q6	Implement a probabilistic discriminative model to predict customer churn from service usage data in a telecom scenario.	5 Marks	6 min
Q7	Apply Bayesian logistic regression to predict whether a customer will purchase a product after viewing online advertisements.	5 Marks	6 min
Q8	Use linear regression to forecast monthly sales for a retail company based on advertising expenditure and seasonal demand.	5 Marks	6 min
Q9	Apply Naïve Bayes classification in a news portal to categorize articles into sports, politics, and entertainment.	5 Marks	6 min
Q10	Use Bayesian logistic regression to predict student placement outcomes based on CGPA, coding skills, and aptitude scores.	5 Marks	6 min

1. Apply a Linear Regression Model to Predict Apartment Rent

Aim

To develop a Linear Regression model that predicts apartment rent based on floor area, number of rooms, and location in a real-estate scenario.

Algorithm: Multiple Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. Multiple Linear Regression uses two or more independent variables to predict a dependent variable.

The model represents the relationship as:

$$Y = b_0 + b_1X_1 + b_2X_2 + b_3X_3$$

Where:

- Y= Predicted apartment rent
- X_1= Floor area
- X_2= Number of rooms
- X_3= Location
- b_0= Intercept
- b_1, b_2, b_3= Model coefficients

Since location is categorical data, it must be converted into a numerical representation before training.

Python Program

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
data = {
    'Area': [500, 600, 700, 800, 900, 1000, 1100, 1200],
    'Rooms': [1, 1, 2, 2, 2, 3, 3, 3],
    'Location': ['Urban', 'Suburban', 'Urban', 'Suburban',
                 'Urban', 'Suburban', 'Urban', 'Suburban'],
    'Rent': [10000, 11000, 14000, 15000, 18000, 19000, 22000, 23000]
}
df = pd.DataFrame(data)
X = df[['Area', 'Rooms', 'Location']]
y = df['Rent']
preprocessor = ColumnTransformer([
   ('location', OneHotEncoder(handle_unknown='ignore'), ['Location'])
], remainder='passthrough')
```

```
model = make_pipeline(preprocessor, LinearRegression())
model.fit(X, y)
new_apartment = pd.DataFrame({
    'Area': [750],
    'Rooms': [2],
    'Location': ['Urban']
})
predicted_rent = model.predict(new_apartment)
print("Apartment Details:")
print(new_apartment)
print("Predicted Monthly Rent: ₹", round(predicted_rent[0], 2))
```

Sample Output

```
Apartment Details:
   Area  Rooms Location
0   750      2    Urban
Predicted Monthly Rent: ₹ 15000.0
```

Result and Interpretation

The Linear Regression model was successfully trained using apartment data containing floor area, number of rooms, location, and monthly rent.

For an apartment with a floor area of 750 square feet, two rooms, and an urban location, the model estimates the monthly rent. The prediction helps property owners and tenants estimate rental prices based on apartment characteristics.

Conclusion: Multiple Linear Regression is suitable for this scenario because it predicts a continuous numerical value using multiple input features. One-hot encoding allows the model to incorporate categorical location information.

2. Linear Classification for Spam Email Detection

Aim

To develop a linear classification model that categorizes emails as spam or non-spam using word frequency features in a mail service system.

Algorithm: Linear Support Vector Machine (SVM)

Linear SVM is a supervised machine learning algorithm used for classification. It finds an optimal hyperplane that separates data into different classes. In this scenario, it classifies emails as spam or non-spam based on their text features.

Input Features and Target

- Input: Email text containing words such as "offer", "free", "win", and "meeting".
- Target: Spam or Non-Spam.

Python Program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
emails = [
    "Win a free prize now",
    "Claim your free gift",
    "Congratulations you won a prize",
    "Meeting scheduled for tomorrow",
    "Please submit your assignment",
    "Let's discuss the project"
]
labels = ["Spam", "Spam", "Spam", "Non-Spam", "Non-Spam", "Non-Spam"]
model = make_pipeline(CountVectorizer(), LinearSVC())
model.fit(emails, labels)
new_emails = [
    "Win a free gift now",
    "Please discuss the project tomorrow"
]
predictions = model.predict(new_emails)
for email, prediction in zip(new_emails, predictions):
    print("Email:", email)
    print("Category:", prediction)
```

Sample Output

```
Email: Win a free gift now
Category: Spam
Email: Please discuss the project tomorrow
Category: Non-Spam
```

Result and Interpretation

The linear SVM model was trained using sample emails labelled as spam or non-spam. CountVectorizer converted the email text into numerical word-frequency features, allowing the classifier to learn patterns associated with each category.

The model classified the first email as spam and the second as non-spam based on the patterns learned from the training data.

Conclusion: Linear SVM is suitable for spam email detection because it learns a decision boundary that separates spam emails from legitimate emails using numerical text features.

3. Naïve Bayes Algorithm for Flu Prediction

Aim

To develop a Naïve Bayes classification model that predicts whether a patient has flu based on symptoms such as fever, cough, and headache.

Algorithm: Naïve Bayes

Naïve Bayes is a supervised machine learning algorithm based on Bayes' theorem. It predicts a class by calculating probabilities from the observed features. It assumes that the features are conditionally independent given the class.

Bayes' theorem is represented as:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

Where $P(C \mid X)$ is the probability of a class given the observed symptoms.

Input Features and Target

- Input: Fever, cough, and headache.
- Target: Flu or No Flu.

Python Program

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
data = {
    'Fever': [1, 1, 0, 1, 0, 0, 1, 0],
    'Cough': [1, 1, 1, 0, 0, 1, 1, 0],
    'Headache': [1, 0, 1, 1, 0, 0, 1, 0],
    'Flu': [1, 1, 0, 1, 0, 0, 1, 0]
}
df = pd.DataFrame(data)
X = df[['Fever', 'Cough', 'Headache']]
y = df['Flu']
model = GaussianNB()
model.fit(X, y)
patient = pd.DataFrame({
    'Fever': [1],
    'Cough': [1],
    'Headache': [1]
})
prediction = model.predict(patient)
if prediction[0] == 1:
    print("Prediction: Flu")
else:
```

```
print("Prediction: No Flu")
```

Sample Output

```
Prediction: Flu
```

Result and Interpretation

The Naïve Bayes model was trained using sample patient data containing fever, cough, and headache symptoms. It predicted whether a patient belonged to the flu or no-flu category.

For a patient experiencing all three symptoms, the model predicted Flu based on the patterns learned from the training data.

Conclusion: Naïve Bayes is suitable for symptom-based classification because it uses probability to estimate the most likely class. This example is for educational purposes and should not be used as a medical diagnosis.

4. Discriminant Functions for Loan Eligibility Classification

Aim

To develop a classification model using discriminant functions to classify loan applicants as eligible or non-eligible based on their income and credit score.

Algorithm: Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis (LDA) is a supervised machine learning algorithm that classifies data by finding combinations of input features that best separate different classes. In this scenario, it uses income and credit score to classify loan applicants.

Input Features and Target

- Input: Applicant income and credit score.
- Target: Eligible or Non-Eligible.

Python Program

```
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
data = {
    'Income': [25000, 30000, 35000, 40000, 50000, 60000, 70000, 80000],
    'CreditScore': [500, 550, 580, 600, 650, 700, 750, 800],
    'Eligibility': ['Non-Eligible', 'Non-Eligible', 'Non-Eligible', 'Non-Eligible',
                    'Eligible', 'Eligible', 'Eligible', 'Eligible']
}
df = pd.DataFrame(data)
X = df[['Income', 'CreditScore']]
y = df['Eligibility']
model = LinearDiscriminantAnalysis()
```

```
model.fit(X, y)
applicant = pd.DataFrame({
  'Income': [65000],
  'CreditScore': [720]
})
prediction = model.predict(applicant)
print("Predicted Loan Eligibility:", prediction[0])
```

Sample Output

```
Predicted Loan Eligibility: Eligible
```

Result and Interpretation

The LDA model was trained using sample loan applicant data containing income, credit scores, and eligibility labels. It learned to distinguish between eligible and non-eligible applicants.

For an applicant with an income of ₹65,000 and a credit score of 720, the model predicted Eligible.

Conclusion: Linear Discriminant Analysis is suitable for loan eligibility classification because it identifies patterns that distinguish between applicant groups using their financial features. Actual loan decisions require additional financial checks and must not rely solely on this illustrative model.

5. Probabilistic Generative Model for Handwritten Digit Recognition

Aim

To develop a probabilistic generative model that recognizes handwritten digits from 0 to 9 using pixel values from digit images.

Algorithm: Gaussian Naïve Bayes

Gaussian Naïve Bayes is a probabilistic generative classification algorithm based on Bayes' theorem. It learns the probability distribution of input features for each class and predicts the most probable class for a given input.

In handwritten digit recognition, the model learns patterns in pixel values associated with each digit and predicts which digit an image represents.

Input Features and Target

- Input: Pixel intensity values of handwritten digit images.
- Target: Digit labels from 0 to 9.

Python Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
digits = load_digits()
X = digits.data
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Actual Digit:", y_test[0])
print("Predicted Digit:", predictions[0])
print("Model Accuracy:", round(accuracy_score(y_test, predictions) * 100, 2), "%")
```

Sample Output

```
Actual Digit: 6
Predicted Digit: 6
Model Accuracy: 84.72 %
```

Result and Interpretation

The Gaussian Naïve Bayes model was trained using handwritten digit images and their corresponding labels. It predicted digit classes based on the pixel intensity patterns learned during training.

The actual and predicted digits can be compared to check whether the model correctly recognizes a handwritten digit. The accuracy score indicates the percentage of test images classified correctly.

Conclusion: Gaussian Naïve Bayes demonstrates probabilistic generative classification by modelling the distribution of pixel features for each digit class. It can be used for basic handwritten digit recognition, although other algorithms may achieve better accuracy.

6. Probabilistic Discriminative Model for Customer Churn Prediction

Aim

To develop a probabilistic discriminative model that predicts whether a customer will leave a subscription service based on service usage, monthly charges, and customer tenure.

Algorithm: Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification. It estimates the probability that an input belongs to a particular class using the logistic function. Unlike generative models, it directly models the relationship between input features and the

target class.

The logistic function is:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-z}}$$

Where $z = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$.

Input Features and Target

- Input: Service usage, monthly charges, and customer tenure.
- Target: Churn or No Churn.

Python Program

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
data = {
    'Usage': [2, 3, 4, 5, 6, 7, 8, 9],
    'MonthlyCharges': [900, 850, 800, 750, 700, 650, 600, 550],
    'Tenure': [2, 4, 6, 8, 12, 18, 24, 36],
    'Churn': [1, 1, 1, 1, 0, 0, 0, 0]
}
df = pd.DataFrame(data)
X = df[['Usage', 'MonthlyCharges', 'Tenure']]
y = df['Churn']
model = LogisticRegression()
model.fit(X, y)
customer = pd.DataFrame({
    'Usage': [3],
    'MonthlyCharges': [880],
    'Tenure': [3]
})
prediction = model.predict(customer)
probability = model.predict_proba(customer)[0][1]
if prediction[0] == 1:
    print("Prediction: Customer Likely to Churn")
else:
    print("Prediction: Customer Not Likely to Churn")
print("Churn Probability:", round(probability * 100, 2), "%")
```

Sample Output

```
Prediction: Customer Likely to Churn
Churn Probability: 100.0 %
```

Result and Interpretation

The Logistic Regression model was trained using sample customer data containing service usage, monthly charges, and tenure. It learned the relationship between these features and customer churn.

For a customer with low usage, high monthly charges, and a short subscription period, the model predicts a higher likelihood of churn based on the patterns in the training data.

Conclusion: Logistic Regression is suitable for customer churn prediction because it estimates the probability of a customer leaving a service and classifies the customer as likely or unlikely to churn.

7. Bayesian Logistic Regression for Advertisement Purchase Prediction

Aim

To develop a Bayesian Logistic Regression model that predicts whether a customer will purchase a product after viewing an advertisement based on advertisement views, clicks, and customer characteristics.

Algorithm: Bayesian Logistic Regression

Bayesian Logistic Regression is a probabilistic classification algorithm that combines Logistic Regression with Bayesian inference. It uses prior distributions for model parameters and updates them using observed data to estimate the probability of a purchase.

The logistic function is:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-z}}$$

where $z = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$.

Unlike ordinary Logistic Regression, Bayesian Logistic Regression estimates a probability distribution over the model parameters rather than only a single set of parameter estimates.

Input Features and Target

- Input: Advertisement views, advertisement clicks, and customer age.
- Target: Purchase or No Purchase.

Python Program

```
import numpy as np
import pymc as pm
X = np.array([
    [1, 0, 22],
    [2, 0, 25],
    [3, 1, 28],
    [4, 1, 32],
    [5, 2, 35],
    [6, 3, 38],
    [7, 4, 42],
```

```
    [8, 5, 45]
], dtype=float)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_scaled = (X - X_mean) / X_std
with pm.Model() as model:
    intercept = pm.Normal("intercept", mu=0, sigma=5)
    coefficients = pm.Normal("coefficients", mu=0, sigma=2, shape=3)
    logits = intercept + pm.math.dot(X_scaled, coefficients)
    probability = pm.Deterministic("probability", pm.math.sigmoid(logits))
    pm.Bernoulli("observed", p=probability, observed=y)
    trace = pm.sample(1000, tune=1000, chains=2, cores=1, random_seed=42,
progressbar=False)
new_customer = np.array([[6, 3, 35]], dtype=float)
new_customer_scaled = (new_customer - X_mean) / X_std
intercept_samples = trace.posterior["intercept"].values.reshape(-1)
coefficient_samples = trace.posterior["coefficients"].values.reshape(-1, 3)
logits = intercept_samples + np.sum(coefficient_samples * new_customer_scaled, axis=1)
purchase_probabilities = 1 / (1 + np.exp(-logits))
purchase_probability = purchase_probabilities.mean()
print("Purchase Probability:", round(purchase_probability * 100, 2), "%")
print("Prediction:", "Purchase" if purchase_probability >= 0.5 else "No Purchase")
```

Sample Output

Purchase Probability: 75.00 %

Prediction: Purchase

Result and Interpretation

The Bayesian Logistic Regression model was trained using sample advertisement and customer data. It estimated the probability that a customer would purchase a product based on advertisement views, clicks, and age.

The model calculates the average predicted purchase probability across posterior samples. If the estimated probability is at least 50%, the customer is classified as likely to purchase; otherwise, the customer is classified as unlikely to purchase.

Conclusion: Bayesian Logistic Regression is suitable for advertisement purchase prediction because it estimates purchase probabilities while accounting for uncertainty in the model parameters.

8. Linear Regression for Monthly Sales Forecasting

Aim

To develop a Linear Regression model that forecasts monthly sales based on advertising expenditure and seasonal demand.

Algorithm: Multiple Linear Regression

Multiple Linear Regression is a supervised machine learning algorithm used to predict a continuous numerical value using two or more independent variables. In this scenario, it estimates monthly sales based on advertising expenditure and seasonal demand.

The equation is:

$$Y = b_0 + b_1X_1 + b_2X_2$$

Where:

- Y= Predicted monthly sales
- X_1= Advertising expenditure
- X_2= Seasonal demand
- b_0= Intercept
- b_1, b_2= Model coefficients

Input Features and Target

- Input: Advertising expenditure and seasonal demand.
- Target: Monthly sales.

Python Program

```
import pandas as pd
from sklearn.linear_model import LinearRegression
data = {
    'Advertising': [10, 15, 20, 25, 30, 35, 40, 45],
    'SeasonalDemand': [2, 3, 2, 4, 3, 5, 4, 5],
    'Sales': [100, 130, 150, 190, 210, 250, 270, 310]
}
df = pd.DataFrame(data)
X = df[['Advertising', 'SeasonalDemand']]
y = df['Sales']
model = LinearRegression()
model.fit(X, y)
new_month = pd.DataFrame({
    'Advertising': [30],
    'SeasonalDemand': [4]
})
predicted_sales = model.predict(new_month)
print("Advertising Expenditure:", new_month['Advertising'].iloc[0])
print("Seasonal Demand:", new_month['SeasonalDemand'].iloc[0])
print("Predicted Monthly Sales:", round(predicted_sales[0], 2))
```

Sample Output

```
Advertising Expenditure: 30
Seasonal Demand: 4
Predicted Monthly Sales: 218.08
```

Result and Interpretation

The Multiple Linear Regression model was trained using sample data containing advertising expenditure, seasonal demand, and sales figures. It learned the relationship between the input features and monthly sales.

For a month with an advertising expenditure of 30 units and a seasonal demand value of 4, the model estimates the expected sales. Businesses can use this prediction to support advertising budgets, inventory planning, and sales forecasting.

Conclusion: Multiple Linear Regression is suitable for monthly sales forecasting because it predicts numerical sales values using multiple influencing factors. However, reliable forecasting requires representative historical data and appropriate seasonal information.

9. Naïve Bayes Algorithm for News Article Classification

Aim

To develop a Naïve Bayes classification model that categorizes news articles into Sports, Politics, or Entertainment based on the words used in the articles.

Algorithm: Multinomial Naïve Bayes

Multinomial Naïve Bayes is a supervised machine learning algorithm based on Bayes' theorem. It is commonly used for text classification because it works with word frequencies to estimate the probability that a document belongs to a particular category.

Bayes' theorem is represented as:

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

Where $P(C \mid X)$is the probability that an article belongs to category C, given its words.

Input Features and Target

- Input: Words and word frequencies in news articles.
- Target: Sports, Politics, or Entertainment.

Python Program

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
articles = [
    "The team won the cricket match",
    "The player scored a goal in football",
    "The government announced new policies",
```

```
    "The election campaign started today",
    "The actor starred in a new movie",
    "The singer released a new album"
]
labels = [
    "Sports",
    "Sports",
    "Politics",
    "Politics",
    "Entertainment",
    "Entertainment"
]
model = make_pipeline(CountVectorizer(), MultinomialNB())
model.fit(articles, labels)
new_articles = [
    "The team won the football match",
    "The government announced election policies",
    "The actor released a new movie"
]
predictions = model.predict(new_articles)
for article, prediction in zip(new_articles, predictions):
    print("News Article:", article)
    print("Category:", prediction)
```

Sample Output

```
News Article: The team won the football match
Category: Sports
News Article: The government announced election policies
Category: Politics
News Article: The actor released a new movie
Category: Entertainment
```

Result and Interpretation

The Multinomial Naïve Bayes model was trained using sample news articles labelled as Sports, Politics, or Entertainment. The CountVectorizer converted the text into numerical word counts, enabling the model to learn associations between words and categories.

The model classified each new article into the most probable category based on the patterns learned during training.

Conclusion: Multinomial Naïve Bayes is suitable for news article classification because it uses word frequencies and probability calculations to categorize text efficiently.

10. Bayesian Logistic Regression for Student Placement Prediction

Aim

To develop a Bayesian Logistic Regression model that predicts whether a student will be placed in a company based on CGPA, coding skills, and aptitude scores.

Algorithm: Bayesian Logistic Regression

Bayesian Logistic Regression is a probabilistic classification algorithm that combines Logistic Regression with Bayesian inference. It uses prior distributions for model parameters and updates them using training data to estimate the probability of a student being placed.

The logistic function is:

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-z}}$$

Where $z = b_0 + b_1X_1 + b_2X_2 + b_3X_3$.

Unlike ordinary Logistic Regression, Bayesian Logistic Regression estimates a probability distribution over the model parameters, allowing uncertainty in the predictions to be considered.

Input Features and Target

- Input: CGPA, coding skills score, and aptitude score.
- Target: Placed or Not Placed.

Python Program

```
import numpy as np
import pymc as pm
X = np.array([
    [6.0, 40, 45],
    [6.5, 45, 50],
    [7.0, 50, 55],
    [7.2, 55, 60],
    [7.5, 65, 65],
    [8.0, 70, 75],
    [8.5, 80, 80],
    [9.0, 90, 90]
], dtype=float)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
X_mean = X.mean(axis=0)
X_std = X.std(axis=0)
X_scaled = (X - X_mean) / X_std
with pm.Model() as model:
    intercept = pm.Normal("intercept", mu=0, sigma=5)
    coefficients = pm.Normal("coefficients", mu=0, sigma=2, shape=3)
    logits = intercept + pm.math.dot(X_scaled, coefficients)
```

```
    probability = pm.Deterministic("probability", pm.math.sigmoid(logits))
    pm.Bernoulli("observed", p=probability, observed=y)
    trace = pm.sample(1000, tune=1000, chains=2, cores=1, random_seed=42,
progressbar=False)
student = np.array([[8.2, 75, 80]], dtype=float)
student_scaled = (student - X_mean) / X_std
intercept_samples = trace.posterior["intercept"].values.reshape(-1)
coefficient_samples = trace.posterior["coefficients"].values.reshape(-1, 3)
logits = intercept_samples + np.sum(coefficient_samples * student_scaled, axis=1)
placement_probabilities = 1 / (1 + np.exp(-logits))
placement_probability = placement_probabilities.mean()
print("Placement Probability:", round(placement_probability * 100, 2), "%")
print("Prediction:", "Placed" if placement_probability >= 0.5 else "Not Placed")
```

Sample Output

Placement Probability: 85.00 %

Prediction: Placed

Result and Interpretation

The Bayesian Logistic Regression model was trained using sample student data containing CGPA, coding skills, aptitude scores, and placement labels.

For a student with a CGPA of 8.2, a coding score of 75, and an aptitude score of 80, the model estimates the probability of placement using the learned parameter distributions. If the estimated probability is at least 50%, the student is classified as placed; otherwise, the student is classified as not placed.

Conclusion: Bayesian Logistic Regression is suitable for student placement prediction because it estimates the probability of placement while accounting for uncertainty in the model parameters. The prediction is an educational demonstration and does not guarantee an actual placement outcome.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage	Level 4 – (Exemplary)	Level 3 – (Accomplished)	Level 2 – (Developing)	Level 1 – (Beginning)
Understanding of Scenario	20%	Demonstrates a comprehensive understanding of the problem and its context.	Demonstrates a clear understanding of the problem.	Demonstrates a partial understanding of the problem.	Demonstrates limited understanding of the problem.
Identification of Key Issues	20%	Identifies all relevant issues and key factors accurately.	Identifies most relevant issues and key factors.	Identifies some relevant issues but misses important factors.	Fails to identify the key issues accurately.
Application of Concepts	25%	Applies relevant concepts accurately and effectively to solve the problem.	Applies relevant concepts correctly with minor errors.	Applies concepts with some errors or gaps.	Demonstrates an incorrect or inadequate application of concepts.
Decision Making	20%	Makes logical, well-supported decisions with clear justification.	Makes appropriate decisions with reasonable justification.	Makes decisions with limited justification.	Makes inappropriate decisions without sufficient justification.
Clarity of Response	15%	Presents a well-organized, clear, and precise response.	Presents a clear and organized response with minor issues.	Presents a partially organized response with some clarity issues.	Presents an unclear, poorly organized response.